

Project

Section 1:

Python Code of Face Recognition System A:

```
import os #This line uses and imports the modules of OS.
import re #This line uses and imports the modules of RE.
from PIL import Image #This line uses PIL and imports its Image module.
import numpy as np #This line uses and imports the modules of Numpy.
import pandas as pd #This line uses and imports the modules of Pandas.
```

```
"""
```

Creation Date: April 25, 2025

@author of Section 1: Chetan Hiremath and Nirupam Dasika

Part I - System_A Implementation (Final Correct Version)

```
"""
```

```
"""
```

These 6 lines that are below this multi-line comment define the numericalfile() function that numerically sorts the files of the folders.

```
"""
```

```
def numericalfile(filename):
```

```
    filenumbermatch = re.search(r'(\d+)', filename) #This line defines the filenumbermatch variable that uses the re.search() function to handle regular expressions and use the pattern for matching digits.
```

```
    #This if-statement with 2 lines checks if the match is valid.
```

```
    if filenumbermatch:
```

```
        return int(filenumbermatch.group(1)) #This line converts the string of digits to an integer.
```

```
    return -1 #This line returns -1 if there is no match.
```

```
"""
```

These 32 lines that are below this multi-line comment define the binarization() function that performs binarization and uses GallerySet and ProbeSet's face images to perform the

comparison of gallery and probe face images and generate the 100 X 100 score matrix and other calculations that are rounded to 2 decimal places by the built-in round() function.

"""

```
def binarization(inputfolder1, inputfolder2):
    os.makedirs(inputfolder1, exist_ok = True) #This line checks if the first folder exists.
    os.makedirs(inputfolder2, exist_ok = True) #This line checks if the second folder exists.
    pgm_files1 = sorted([f for f in os.listdir(inputfolder1) if f.endswith('.pgm')], key =
numericalfile)[:100] #This line defines the pgm_files1 variable that gets the first 100 files of the
first folder, filters them if the filenames end with '.pgm', and sorts them numerically by using
the numericalfile() function.
    pgm_files2 = sorted([f for f in os.listdir(inputfolder2) if f.endswith('.pgm')], key =
numericalfile)[:100] #This line defines the pgm_files2 variable that gets the first 100 files of the
second folder, filters them if the filenames end with '.pgm', and sorts them numerically by using
the numericalfile() function.
    threshold = 128 #This line defines the threshold variable that is used for thresholding.
    gallery_images = [(np.array(Image.open(os.path.join(inputfolder1, f)).convert('L')) >
threshold).astype(np.uint8).flatten() for f in pgm_files1] #This line defines the gallery_images
variable that uses thresholding and converts the files of the first folder to binary images, which
are converted to 1D arrays. If the grayscale pixel is greater than the threshold, then the output
binary pixel is 1. If not, then the output binary pixel is 0.
    probe_images = [(np.array(Image.open(os.path.join(inputfolder2, f)).convert('L')) >
threshold).astype(np.uint8).flatten() for f in pgm_files2] #This line defines the probe_images
variable that uses thresholding and converts the files of the second folder to binary images,
which are converted to 1D arrays. If the grayscale pixel is greater than the threshold, then the
output binary pixel is 1. If not, then the output binary pixel is 0.
    scorematrix = np.zeros((len(gallery_images), len(probe_images))) #This line defines the
scorematrix variable that creates a 2D matrix of the given shape with zeros.
    pixelcount = gallery_images[0].size #This line defines the pixelcount variable that finds the
number of pixels of the gallery face image.
    #The nested for-loop with 4 lines enumerates or iterates to generate gallery and probe face
images' respective comparisons' scores that are stored in the score matrix.
    for i, probe_image in enumerate(probe_images):
        for j, gallery_image in enumerate(gallery_images):
            hammingdistancescore = (np.sum(gallery_image != probe_image) / pixelcount) * 100
#This line defines the hammingdistancescore variable that finds the number of positions of
different pixels of gallery and probe face images.
```

scorematrix[i, j] = hammingdistancescore #This line uses the hammingdistancescore variable to find the respective scores of the hamming distances of gallery and probe face images.

df = pd.DataFrame(np.round(scorematrix, 2), index = pgm_files2, columns = pgm_files1) #This line defines the df variable that creates an organized score matrix with labels.

print("Score Matrix:") #This line prints the "Score Matrix:" message.

print(df) #This line prints the organized 2D score matrix.

genuinescores = np.diag(df.values) #This line defines the genuinescores variable that determines the genuine scores of the gallery and probe face images of the same subjects by finding the diagonal values of the score matrix.

imposterscores = df.values[~np.eye(df.values.shape[0], dtype = bool)] #This line defines the imposterscores variable that determines the imposter scores of the gallery and probe face images of the different subjects by finding the non-diagonal values of the score matrix.

decidabilityindexvalue = (np.sqrt(2) * (np.abs(np.mean(genuinescores) - np.mean(imposterscores)))) / (np.sqrt((np.std(genuinescores) ** 2) + (np.std(imposterscores) ** 2))) #This line defines the decidabilityindexvalue variable that uses the decidability index value formula to calculate the decidability index value.

print("Genuine scores:", genuinescores) #This line prints the genuine scores of the score matrix.

print("Imposter scores:", imposterscores) #This line prints the imposter scores of the score matrix.

print("Decidability Index Value of System_A =", round(decidabilityindexvalue, 2)) #This line prints the decidability index value of the score matrix.

facerecdecidabilityindexvalue = 3.5 #This line defines the facerecdecidabilityindexvalue variable that uses 3.5 as the decidability index value of FaceRec.

#The if-statement with 4 lines checks if the decidability index value of this system is greater than the decidability index value of FaceRec.

if decidabilityindexvalue > facerecdecidabilityindexvalue:

print("System_A's performance is better than FaceRec's performance since", round(decidabilityindexvalue, 2), "or System_A's decidability index value is greater than", facerecdecidabilityindexvalue, "or FaceRec's decidability index value.") #This line prints this message if the decidability index value of this system is greater than the decidability index value of FaceRec.

else:

print("System_A's performance is worse than FaceRec's performance since", round(decidabilityindexvalue, 2), "or System_A's decidability index value is less than", facerecdecidabilityindexvalue, "or FaceRec's decidability index value.") #This line prints this

message if the decidability index value of this system is less than the decidability index value of FaceRec.

```
df.to_csv("scorematrix.csv") #This line writes the score matrix that is saved in the scorematrix.csv file.
```

```
return round(decidabilityindexvalue, 2) #This line returns the decidability index value of this system.
```

```
inputfolder1 = "./GallerySet" #This line defines the inputfolder1 variable that uses the GallerySet folder.
```

```
inputfolder2 = "./ProbeSet" #This line defines the inputfolder2 variable that uses the ProbeSet folder.
```

```
decidabilityindexvalueA = binarization(inputfolder1, inputfolder2) #This line defines the decidabilityindexvalueA variable that uses the binarization() function to perform the comparison of gallery and probe face images by using the GallerySet folder and the ProbeSet folder for thresholding and hamming distance and stores the decidability index value of this system for future calculations.
```

```
print() #This line creates a new line to organize the results.
```

1.

Score Matrix Snippet:

	subject1_img1.pgm	subject2_img1.pgm	subject3_img1.pgm	subject4_img1.pgm	subject5_img1.pgm	subject6_img1.pgm	subject7_img1.pgm	subject8_img1.pgm	subject9_img1.pgm	subject10_img1.pgm
subject1_img2.pgm	19.68	33.48	36.64	37.04	31.64	33.32	28.12	32.32	19.92	41.64
subject2_img2.pgm	29.92	12.36	30.64	28.48	24.6	29.72	33.64	25.92	26.08	30.28
subject3_img2.pgm	32.36	27.76	16.92	19.16	32.32	17.68	37.68	20.04	35.08	22.56
subject4_img2.pgm	33.12	23.4	23.52	11.68	25.72	21.4	28.68	22.0	30.48	22.84
subject5_img2.pgm	32.88	22.92	22.72	23.36	16.04	27.24	31.08	16.72	24.72	25.08
subject6_img2.pgm	26.28	27.6	21.8	21.8	32.4	5.68	30.0	21.56	31.4	27.76
subject7_img2.pgm	29.32	31.68	36.28	31.24	30.96	31.2	7.6	32.76	26.36	34.72
subject8_img2.pgm	27.72	25.2	19.08	20.92	24.32	20.96	30.96	6.36	25.72	27.84
subject9_img2.pgm	23.48	21.36	30.76	28.04	22.4	27.12	24.48	22.6	11.72	34.08
subject10_img2.pgm	38.88	28.04	23.28	21.84	25.08	25.32	33.32	23.12	37.12	7.8

2.

The decidability index value of the score matrix of System_A is 2.91 due to the calculation by the decidability index value formula of this Python Code.

3.

A face recognition system FaceRec has a decidability index value of 3.5. The decidability index value of System_A is 2.91, which is calculated by this Python Code and mentioned in the previous question. If these values are compared to determine the system with the best performance, then System_A's performance is not better than FaceRec's performance since System_A's decidability index value is less than FaceRec's decidability index value. If a decidability index value is high, then it can separate the distribution of the genuine scores and the imposter scores of the face system. If not, then it will cause an overlap between these scores. FaceRec has a high decidability index value that will allow this system to differentiate

between genuine scores and imposter scores easily. But System_A uses Binarization by thresholding to generate a low decidability index value that won't help this system to differentiate between genuine scores and imposter scores efficiently. Binarization doesn't find valuable face information since it has black and white colors only. Therefore, FaceRec is better and more efficient than System_A for face recognition.

Section 2:

Python Code of Face Recognition System_B:

```
#!/usr/bin/env python3
```

```
-- coding: utf-8 --
```

```
""" Created on Apr 28, 2025 @author of Section 2: johnpaulmbagwu and Michael Chukwuka
```

```
Part II - System_B Implementation (Final Correct Version)
```

```
"""
```

Import required libraries

```
import os # For file and directory operations
import numpy as np # For numerical operations and array handling
import cv2 # For image loading and processing from sklearn.cluster
import KMeans # For unsupervised clustering
from scipy.spatial.distance import hamming # For computing Hamming distance
import matplotlib.pyplot as plt # For plotting histograms
```

Define paths to gallery and probe image sets

```
gallery_path = "./GallerySet"
probe_path = "./ProbeSet"
```

Step 1: Load and preprocess images

```
def load_image(path): """ Load an image in grayscale and apply histogram equalization to
enhance contrast and normalize lighting conditions. """ img = cv2.imread(path,
cv2.IMREAD_GRAYSCALE) img = cv2.equalizeHist(img) # Enhance contrast return img

def kmeans_binarize(img): """ Use KMeans clustering with 2 clusters to binarize the image. This
provides a consistent method to threshold the image without a fixed value. """ pixels =
img.reshape(-1, 1) # Flatten image to a column vector kmeans = KMeans(n_clusters=2,
random_state=42, n_init=10).fit(pixels) labels = kmeans.labels_.reshape(img.shape) # Reshape
labels back to image shape

# Ensure high-intensity pixels are mapped to 1 (foreground)
centers = kmeans.cluster_centers_.flatten()
high_intensity_label = np.argmax(centers)
binary_img = (labels == high_intensity_label).astype(np.uint8)
return binary_img

def preprocess_dataset(folder_path): """ Load and binarize all images in a given folder. Returns
a list of binary images. """
    files = sorted(os.listdir(folder_path)) # Sort to ensure consistent order
    binarized_images = []
    for filename in files:
        img = load_image(os.path.join(folder_path, filename)) # Load and normalize
        bin_img = kmeans_binarize(img) # Binarize with KMeans
        binarized_images.append(bin_img) return binarized_images
```

Step 2: Apply preprocessing to both gallery and probe sets

```
gallery_images = preprocess_dataset(gallery_path)
probe_images = preprocess_dataset(probe_path)
```

Step 3: Compute Hamming distances and build score matrix B

```
def build_score_matrix(probe_images, gallery_images): """ Build a score matrix where entry
B[i, j] is the Hamming distance between the i-th probe image and j-th gallery image. """
```

```
n_probe = len(probe_images)
n_gallery = len(gallery_images)
```

```
B = np.zeros((n_probe, n_gallery))
for i in range(n_probe):
    probe_bin = probe_images[i]
    for j in range(n_gallery):
        gallery_bin = gallery_images[j]
        # Compute Hamming distance between flattened binary images
        B[i, j] = hamming(probe_bin.flatten(), gallery_bin.flatten())
return B
```

Compute and save the score matrix

```
B = build_score_matrix(probe_images, gallery_images)
np.save("score_matrix_B.npy", B) # Save the matrix for later use
```

Display a portion of the score matrix for inspection

```
print("Score Matrix B[0:9, 0:9]:\n", B[0:10, 0:10])
```

Step 4: Calculate the Decidability Index d'

Genuine scores: diagonal elements (same subject)

```
genuine_scores_B = np.diag(B)
```

Impostor scores: all non-diagonal elements

```
impostor_scores_B = B[~np.eye(100, dtype=bool)] # Exclude diagonal
```

Compute means and standard deviations

```
mu_g = np.mean(genuine_scores_B)
mu_i = np.mean(impostor_scores_B)
std_g = np.std(genuine_scores_B)
std_i = np.std(impostor_scores_B)
```

Calculate d' -prime: a measure of separability between genuine and impostor scores

```
d_prime_B = abs(mu_g - mu_i) / np.sqrt(0.5 * (std_g2 + std_i2))
print(f"\nDecidability Index ( $d'$ ) for System_B: {d_prime_B:.3f}")
```

Step 5: Calculate improvement over a baseline System_A

```
d_prime_A = decidabilityindexvalueA # <-- Replace with actual value from System_A
delta_d_prime = d_prime_B - d_prime_A
```



```
improvement_factor = round(delta_d_prime, 2) print(f"Improvement Factor (IF):  
{improvement_factor}")
```

Step 6: Plot histogram of genuine and impostor score distributions

```
plt.figure(figsize=(8, 5))  
plt.hist(impostor_scores_B, bins=50, alpha=0.6, label='Impostor Scores', color='red',  
density=True)  
plt.hist(genuine_scores_B, bins=50, alpha=0.6, label='Genuine Scores', color='blue',  
density=True)  
plt.title("System_B: Score Distributions")  
plt.xlabel("Hamming Distance")  
plt.ylabel("Density")  
plt.legend()  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

4.

Score Matrix Snippet:

```
Score Matrix B[0:9, 0:9]:  
[[0.1528 0.306  0.2912 0.292  0.2756 0.3168 0.328  0.3104 0.3464 0.3164]  
 [0.2688 0.078  0.3056 0.292  0.3468 0.2352 0.2544 0.2984 0.2816 0.4156]  
 [0.2672 0.302  0.1168 0.2448 0.306  0.3256 0.3144 0.316  0.276  0.3564]  
 [0.2628 0.3552 0.2396 0.122  0.2328 0.2708 0.362  0.2068 0.2444 0.2552]  
 [0.2688 0.3876 0.2656 0.2088 0.0676 0.2608 0.3088 0.2376 0.2744 0.1644]  
 [0.2732 0.3144 0.2884 0.198  0.2216 0.19  0.3124 0.2404 0.2796 0.2536]  
 [0.2168 0.27  0.2912 0.22  0.262  0.2256 0.2408 0.2464 0.2832 0.2908]  
 [0.2312 0.3436 0.3008 0.1928 0.226  0.2776 0.3336 0.1024 0.2984 0.2412]  
 [0.3308 0.3232 0.2588 0.2124 0.3064 0.3132 0.4204 0.2788 0.1628 0.3144]  
 [0.3076 0.4304 0.3276 0.2364 0.1728 0.2852 0.3444 0.242  0.298  0.1168]]
```

5.

The decidability index value of the score matrix of System_B is 2.911 due to the calculation by the decidability index value formula of this Python Code.

6.

The improvement factor of the score matrices of System_A and System_B is $IF = \text{round}((d' \text{ of System}_B - d' \text{ of System}_A), 2) = \text{round}((2.911 - 2.91), 2) = \text{round}(0.001, 2) = 0.0$ due to the calculation by the improvement factor formula of this Python Code. This value proves that there is no improvement of System_B even though it uses K-Means Clustering Algorithm.

7.

K-Means Method and Interpretation in System_B

In the context of image preprocessing for biometric recognition, consistent and effective image binarization is essential to ensure accurate matching between gallery and probe sets. The **K-Means clustering algorithm** is employed in *System_B* as a robust and adaptive binarization technique that transforms grayscale images into binary format.

The **K-Means algorithm** is an unsupervised learning method that partitions data into k clusters based on similarity—in this case, clustering pixel intensities in a grayscale image. By setting $k = 2$, the algorithm segments the image into two primary intensity groups: one representing the foreground and the other the background. This eliminates the need for a fixed global threshold, which can be unreliable under varying lighting conditions.

After clustering, the group with higher intensity is consistently mapped to 1 and the other to 0, ensuring uniform binary image representation across different images. This process significantly improves the reliability of Hamming distance calculations between binary images, which is central to the matching and recognition pipeline in *System_B*.

Thus, by leveraging K-Means for binarization, the system gains adaptability, consistency, and improved recognition performance, contributing to the overall enhancement of the biometric matching process.